

Estruturas de Decisão

Ensinando o Computador a Escolher Caminhos

Decisões expressas com o Operador Ternário, bifurcações de fluxo com if e else, encadeamentos e menus estruturados com switch...case.

Docente: Jeronimo Silva · Turma 1082 · Senac Centro (Tarde · 2026)

O Que Faremos Hoje

1. Decisões Rápidas (Ternário)

O atalho `condição ? sim : não` : como escolher entre dois valores em apenas uma linha limpa.

2. Blocos de Código (if & else)

Executando sequências completas de comandos apenas quando a condição lógica for atendida.

3. Múltiplos Caminhos (else if)

Classificando faixas de dados: sistema de notas acadêmicas, faixas etárias e limites operacionais.

4. Menus Estruturados (switch...case)

Tratando escolhas de menu do terminal com clareza, utilizando `case`, `break` e `default`.

Integração com Codespaces: Todos os exemplos serão alimentados via `@inquirer/prompts` direto no terminal.

CHECKLIST DE CONCEITOS

Termos e Palavras-Chave da Aula

Palavra-Chave	O que significa na prática?
Operador Ternário	<code>condição ? valorSeTrue : valorSeFalse</code> (a decisão expressa de 1 linha só).
<code>if</code> (Se)	Executa o bloco de código <code>{ ... }</code> apenas se a expressão resultar em <code>true</code> .
<code>else</code> (Senão)	O caminho alternativo obrigatório quando a condição do <code>if</code> resulta em <code>false</code> .
<code>else if</code>	Encadeia uma nova condição para ser testada se a anterior for falsa.
<code>switch</code>	Avalia uma variável contra múltiplos valores constantes pré-definidos de um menu.
<code>case</code> & <code>break</code>	O valor a casar e a ordem expressa de interromper a execução do switch.
<code>default</code>	A rota padrão de segurança do switch quando nenhuma das opções coincide.
Barre o Erro Primeiro	Testar os impedimentos antes para evitar aninhamento de código.

O Operador Ternário (? :)

A Menor e Mais Elegante Forma de Escolher um Valor

Quando precisamos apenas definir **um valor entre duas opções** para uma variável, não é necessário abrir blocos inteiros de chaves. O Ternário resolve em uma linha:

```
// Sintaxe: const resultado = condicao ? valorSeTrue : valorSeFalse;  
  
const idade = 20;  
const statusEleitor = (idade ≥ 18) ? "Voto Obrigatório" : "Facultativo ou Inapto";  
  
console.log(statusEleitor); // Saída: "Voto Obrigatório"
```

💡 O JavaScript avalia a pergunta antes do `?`. Se der `true`, pega o valor à esquerda dos dois pontos (`:`). Se der `false`, pega o da direita.

O Comando `if` (Tomando Uma Decisão)

O comando `if` abre um bloco de chaves `{ ... }` que só é executado se a condição for estritamente verdadeira:

```
import { number } from '@inquirer/prompts';

const velocidade = await number({ message: 'Velocidade do veículo (km/h):' });

// Só executa o bloco interno se velocidade for MAIOR que 60:
if (velocidade > 60) {
  console.log("🚨 ALERTA: Você ultrapassou o limite da via!");
  console.log("📡 Notificação de trânsito enviada ao sistema.");
}

console.log("Obrigado por dirigir com atenção.");
```

Se `velocidade ≤ 60`, todas as linhas dentro das chaves `{ }` são simplesmente ignoradas pelo Node.js.

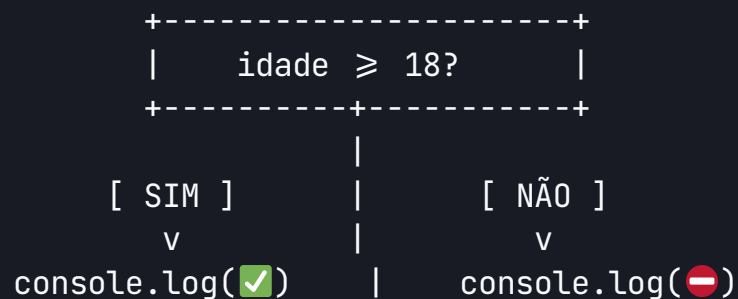
O Comando `else` (O Caminho Alternativo)

Quando precisamos que uma de duas rotas aconteça obrigatoriamente:

```
import { number } from '@inquirer/prompts';

const idade = await number({ message: 'Digite sua idade:' });

if (idade ≥ 18) {
  console.log("✅ Entrada liberada: Bem-vindo ao evento.");
} else {
  console.log("❌ Entrada bloqueada: Evento restrito para maiores de 18 anos.");
}
```



Ternário vs. if / else tradicional

Veja a diferença prática entre as duas abordagens para atribuição de valor:

Com if / else tradicional (6 linhas)

```
let frete;

if (totalCompra ≥ 150) {
  frete = 0;
} else {
  frete = 20;
}
```

Com Operador Ternário (1 linha só!)

```
// Direto, imutável e declarativo:
const frete = (totalCompra ≥ 150) ? 0 : 20;

console.log(`Frete: R$ ${frete}`);
```

Regra Prática: Use o **Ternário** para definir um valor simples entre duas opções. Use **if / else** quando precisar executar comandos complexos, rotas longas ou funções.

Encadeando Cenários com `else if`

Quando temos 3 ou mais faixas de classificação (ex: sistema de notas do Senac):

```
import { number } from '@inquirer/prompts';

const media = await number({ message: 'Média final do aluno (0 a 10):', step: 'any' });

if (media ≥ 7.0) {
  console.log("🎉 Aprovado com Louvor!");
} else if (media ≥ 5.0) {
  console.log("⚠ Em Recuperação: Agende a prova final com a coordenação.");
} else {
  console.log("❌ Reprovado: Necessário refazer a Unidade Curricular.");
}
```

📌 **Atenção à Ordem:** O JavaScript avalia de cima para baixo. Ao encontrar a primeira condição verdadeira, ele executa e **abandona a cadeia inteira!**

O Comando `switch...case`

Quando comparamos uma mesma variável com valores fixos de um menu:

```
import { select } from '@inquirer/prompts';

const plano = await select({
  message: 'Escolha seu plano de suporte:',
  choices: [
    { name: 'Bronze (Suporte por e-mail)', value: 'B' },
    { name: 'Prata (Suporte por chat)', value: 'P' },
    { name: 'Ouro (Suporte 24h dedicado)', value: 'O' }
  ]
});

switch (plano) {
  case 'B':
    console.log("Plano Bronze selecionado: R$ 49,90/mês.");
    break; // Impede que o fluxo caia nos casos abaixo!
  case 'P':
    console.log("Plano Prata selecionado: R$ 99,90/mês.");
    break;
  case 'O':
    console.log("Plano Ouro selecionado: R$ 199,90/mês.");
    break;
  default:
    console.log("Opção desconhecida.");
}
```

O Perigo do `break` no `switch`

⚠ O Fenômeno do "Fall-Through": Se você esquecer o comando `break`, o JavaScript continuará executando os casos seguintes mesmo que eles não correspondam!

```
const diaSemana = 1;

switch (diaSemana) {
  case 1:
    console.log("Domingo"); // Sem break!
  case 2:
    console.log("Segunda-feira"); // VAI EXECUTAR TAMBÉM!
    break;
}

// Saída impressa no terminal:
// Domingo
// Segunda-feira
```

Lembre-se: Use `break` sempre ao final de cada `case`, a menos que deseje agrupar opções propositalmente.

Código Limpo: Evite o "Código Espaguete"

Evite aninhar muitos `if` dentro de outros `if`. Prefira barrar os impedimentos primeiro:

❌ Aninhamento Excessivo (Pirâmide)

```
if (usuarioLogado) {  
  if (temSaldo) {  
    if (produtoEmEstoque) {  
      console.log("✅ Compra realizada!");  
    }  
  }  
}
```

✅ Barre o Erro Primeiro (Linear)

```
if (!usuarioLogado) {  
  console.log("❌ Erro: Usuário não logado.");  
} else if (!temSaldo) {  
  console.log("❌ Erro: Saldo insuficiente.");  
} else if (!produtoEmEstoque) {  
  console.log("❌ Erro: Produto sem estoque.");  
} else {  
  console.log("✅ Compra realizada com sucesso!");  
}
```

Desafio: Calculadora de Desconto Comercial

Crie um arquivo chamado `calculadora_desconto.js` com o seguinte fluxo:

1. Peça o **valor total da compra** via `number({ min: 0, step: 'any' })`.
2. Peça a **forma de pagamento** via `select()` :
 - PIX (10% de desconto)
 - Cartão à Vista (5% de desconto)
 - Cartão Parcelado (Sem desconto)
3. Use `switch...case` para definir a porcentagem de desconto.
4. Calcule o valor final e imprima um resumo formatado.

Desafio Extra: Se o valor final for superior a R\$ 200,00, use o **operador ternário** para exibir a mensagem: " Parabéns! Você ganhou um cupom de R\$ 20 para a próxima compra."

Desafio Final: O Caixa Eletrônico Senac

Crie o arquivo `caixa_eletronico.js` simulando uma operação bancária:

1. Defina um saldo fixo: `let saldo = 1500.00; .`
2. Pergunte via `number()` o valor que o usuário deseja sacar.
3. Aplique as regras com `if / else if / else` :
 - Se o valor for menor ou igual a 0: Imprima *"Valor inválido."*
 - Se o valor for maior que o saldo: Imprima *"Saldo insuficiente."*
 - Se o valor não for múltiplo de 10 (`valor % 10 !== 0`): Imprima *"Notas disponíveis apenas de R\$ 10, 20, 50 e 100."*
 - Caso passe em todas as regras: Subtraia o valor do saldo e mostre *"Saque efetuado com sucesso! Saldo restante: R\$..."*

SÍNTESE & ENCERRAMENTO

O Que Dominamos na Aula 03?

1. Decisões Expressas

Atribuições rápidas de valores em uma linha só com o Operador Ternário.

2. Blocos & Bifurcações

Controle de fluxo condicional com `if`, `else` e múltiplos ramos com `else if`.

3. Menus & Boas Práticas

Decisões limpas com `switch`, blindagem com `break` e código sem aninhamento excessivo.

PRÓXIMA AULA · AULA 04

Estruturas de Repetição

O Poder da Automação com Laços

Repetindo tarefas sem cansar a máquina: while, do...while, for clássico e o moderno for...of em coleções de dados.

Parabéns pelo avanço e até a próxima aula! 🚀